

KAZAKHSTAN DEEPTECH INNOVATION

Young Kazah Language

Суверенный системный язык программирования следующего поколения: **0 байт GC**, нативная аппаратная скорость и **математическая безопасность памяти** без барьера входа.

- 0-Byte Heap Overhead
- NLL Static Borrow Checker
- Go-Like Ergonomics 2.0
- Native AOT MSVC & MIR VM

Айтжан Али (14 лет)
Solo DeepTech Developer & Compiler Architect

АНАЛИЗ РЫНКА И ВЫЗОВЫ

Фундаментальный тупик системного ПО

Существующие языки заставляют инженеров идти на критические компромиссы

C / C++

Скорость без защиты. Прямой доступ к регистрам процессора, но ручное управление памятью.

- **70% всех CVE в мире** (Google/MS) — это баги памяти (Use-After-Free, buffer overflow).
- Катастрофический риск в авионике, дронах и роботах.

Go / Java / C#

Безопасность ценой пауз. Защита памяти делегирована Garbage Collector (GC).

- **Stop-The-World паузы:** непредсказуемый джиттер, срыв тактов в жестком реальном времени (Hard RT).
- Огромный оверхед RAM: непригодны для микроконтроллеров с 16–64 КБ ОЗУ.

Rust

Безопасность ценой стены сложности.
заимствований без GC.

- **Экстремальный барьер входа:** ручные
  перегруженный синтаксис.
- Массовый отток инженеров, медленный

АРХИТЕКТУРНЫЙ ПРОРЫВ

Решение: Архитектура YKL 2.0

Скорость Си + Безопасность Rust + Лаконичность Go

Zero-GC & Hard RT

100% стековое управление памятью.

- Никаких пауз сборщика мусора: детерминированное время исполнения каждого такта.
- 0 байт служебных заголовков: плоский layout структур в ОЗУ.
- Идеально для Bare-Metal прошивок (STM32, RISC-V).

Математический NLL

Статический Borrow Checker на MIR CFG.

- Формальный анализ потока данных (Points-To Dataflow).
- Ошибки UseAfterMove и BorrowConflict ловятся *до компиляции*.
- Non-Lexical Lifetimes: ссылки живут ровно столько, сколько используются.

Эргономика 2.0

Минимальный порог входа в системн

- Короткое объявление `:=` с автовыводом
- Методы прямо внутри `struct` (без гром
- **Auto-borrowing:** компилятор выводит с

ЭРГОНОМИКА И СИНТАКСИС

Сравнение синтаксиса: YKL 2.0 vs Rust

Снижение когнитивной нагрузки на инженера при сохранении 100% гарантий безопасности

СЛОЖНЫЙ СИНТАКСИС: RUST — РУЧНЫЕ ЛАЙФТАЙМЫ & ПЕРЕГРУЖЕННОСТЬ

```
// Rust: разрозненные структуры и impl-блоки struct Account { balance: i32, } impl Account {
fn withdraw(&mut self, amt: i32) -> bool { if self.balance < amt { return false; }
self.balance -= amt; true } } // Ручные лайфтаймы в сигнатурах: fn transfer<'a, 'b>( from:
&'a mut Account, to: &'b mut Account, amt: i32 ) { ... } // Вызов с обязательным &mut:
transfer(&mut acc1, &mut acc2, 500);
```

ЧИСТЫЙ СИНТАКСИС: YKL 2.0 — A

```
// YKL 2.0: методы прямо внутри struct struct Account { balance: int
amt: int) -> bool { if self.balance < amt { return false } self.bala
return true } } // Без ручных 'a: компилятор докажет безопасность fn
Account, to: &mut Account, amt: int) { ... } // Auto-borrowing: пере
ручных &mut acc1 := Account { balance: 15000 } acc2 := Account { bal
transfer(acc1, acc2, 3500)
```

ИНЖЕНЕРНАЯ БАЗА

Архитектура компилятора: 8 стадий конвейера

16 модульных крейтов на Rust по стандартам Clang и Rustc

01

apl_lexer

Токенизация исходников, прецизионные спаны строк и колонок для сообщений об ошибках.

02

apl_parser

Рекурсивный спуск + Pratt Parsing для приоритетов выражений. YKL 2.0 грамматика.

03

apl_sema

Разрешение типов, проверка полей структур и дружелюбная менторская диагностика.

04

apl_hir

High-Level IR: очистка синт
сигнатур и выражений.

05

apl_mir

Control Flow Graph (CFG) из базовых блоков (BasicBlock), Statements и Terminators.

06

apl_borrowck

Ядро безопасности: Non-Lexical Lifetimes Dataflow анализ графа владения и точек заимствования.

07

apl_vm

Интерпретатор MIR для мгновенного REPL-тестирования без задержек сборки.

08

apl_codegen

Оптимизирующий AOT бэ
максимальной скорости.

🛠 **Встроенный LSP сервер:** `apl_lsp` (автодополнение и диагностика в VS Code)

📐 **Форматтер кода:** `apl_fmt` (автоматический код-стайл)

🧪 **Тестовый фреймворк:** `apl_`

ФЛАГМАНСКИЙ КЕЙС / HARD REAL-TIME

Полетный контроллер дрона (БПЛА)

Стабилизация высоты полета в реальном времени: 0 байт в куче, детерминированная физика

Инженерные требования авионики

В автономных БПЛА любая пауза сборщика мусора (GC) приводит к срыву ШИМ-сигнала моторов и падению аппарата.

- **Детерминированная память на стеке:** `DroneState` занимает ровно **12 байт**, `PidController` — **24 байта**. 0 байт аллокаций в куче!
- **Фиксированная точка (Scale x1000):** расчеты без FPU — работает на микроконтроллерах за $0.5 (STM32F0, AVR, RISC-V).
- **Anti-Windup Clamping:** защита интегратора от перенасыщения, аппаратное ограничение ШИМ 0..1000.

✅ **Итог верификации компилятора:**

Код `drone_pid_controller.yk` выдает **бит-в-бит идентичный результат** в виртуальной машине и нативном бинарнике `app.exe`.

DRONE_PID_CONTROLLER.YK (ФРАГМЕНТ)

```
struct PidController { kp: int, ki: int, kd: int integral_sum: int prev_er
self, drone: &DroneState, target: int) -> int { error := target - drone.al
* error) / 1000 // Anti-windup clamping self.integral_sum = self.integral_
self.integral_sum > 30000 { self.integral_sum = 30000 } i_term := (self.ki
d_term := (self.kd * (error - self.prev_error)) / 1000 self.prev_error = e
+ i_term + d_term if raw_pwm > 1000 { return 1000 } return raw_pwm } }
```

ДАННЫЕ ТЕСТИРОВАНИЯ

Телеметрия симуляции: Физика полета дрона

25 тактов управления по 100 мс: выход на 1000 мм, демпфирование перелета и удержание

ЦЕЛЕВАЯ ВЫСОТА	ПИКОВАЯ ВЫСОТА	СТАБИЛИЗАЦИЯ	КУЧА (HEAP ALLOCATED)
1 000 мм	1 146 мм	1 124 мм / −93 мм/с	0 байт



ИНТЕЛЛЕКТУАЛЬНАЯ ДИАГНОСТИКА

Borrow Checker как дружелюбный ментор

Компилятор YKL не ставит в тупик, а обучает инженера правильной архитектуре

Диагностика конфликтов заимствований

Вместо сухих кодов ошибок YKL выдает структурированные подсказки на русском языке с указанием точного места и способа решения.

```
❌ Ошибка заимствования [BorrowConflict] на строке 3: Конфликт изменяемой (&mut) и неизменяемой (&) ссылки. 💡 Подсказка: Завершите чтение по старой ссылке перед изменением объекта.
```

```
❌ Ошибка памяти [UseAfterMove] на строке 4: Переменная используется после передачи владения (Move). 💡 Подсказка: Чтобы продолжить пользоваться объектом после вызова, передавайте его по ссылке (&).
```

Non-Lexical Lifetimes (NLL) в действии

В отличие от наивных компиляторов, заимствование освобождается сразу после п закрывающей скобке }.

```
fn main() -> int { x := 10 // y заимствует x мутабельно: y := &mut x
видит: y больше нигде не читается! // Заимствование автоматически сн
42 // Безопасно разрешено без конфликта! println(x) return 0 }
```

КОНКУРЕНТНОЕ ПРЕИМУЩЕСТВО

Большая сравнительная матрица языков

Позиционирование YKL относительно мировых лидеров индустрии

Критерий	YKL (Наш проект)	C / C++	Rust	Go (Golang)
Безопасность памяти	✓ **Статический NLL**	✗ Ручной free/malloc	✓ **Borrow Checker**	✓ **За счет GC**
Сборщик мусора (GC)	✓ **НЕТ (0 байт GC)**	✓ **НЕТ**	✓ **НЕТ**	✗ ДА (Stop-The-World)
Порог входа / Кривая обучения	✓ **Низкий (Go-like)**	Низкий (но опасный)	✗ Экстремально высокий	✓ **Низкий**
Ручные лайфтаймы <'a>	✓ **НЕТ (Авто-вывод)**	НЕТ (нет проверок)	✗ ДА (Обязательны)	НЕТ
Жесткое реальное время (Hard RT)	✓ **100% готов**	✓ **Стандарт**	✓ **Готов**	✗ Непригоден
Накладные расходы ОЗУ	✓ **0 байт (Плоский layout)**	0 байт	0 байт	✗ 2–3x на служебные данные
Автономность сборщика	✓ **Нативный CLI**	Нативный	Нативный	Нативный

ТЕКУЩАЯ ЗРЕЛОСТЬ

Инженерное качество и результаты

Проект прошел строгое автоматическое тестирование на всех уровнях стека



16 Модульных Крейтов

Чистая компонентная архитектура: компилятор разбит на независимые библиотеки от лексера до генератора C11 и LSP.



100% Тестов Пройдено

`cargo test --workspace` и `python run_tests.py` проходят с нулевым количеством сбоев. Проверены все сценарии.



Автономный Native Build

Никаких Docker-контейнеров или виртуа
готовые бинарники `app.exe` через станд

СТРАТЕГИЯ МАСШТАБИРОВАНИЯ

Дорожная карта развития (2026–2027)

От авторского компилятора — к признанному международному стандарту

Q4 2026 (ТЕКУЩАЯ)

Стабилизация YKL 2.0

- Эргономичный синтаксис (`:=`, методы в структурах, auto-borrowing).
- Флагманские примеры: авионика БПЛА, финтех и численные методы.
- Полноценная интеграция с MSVC AOT.

Q1 2027

Embedded & Пакеты

- Прямой LLVM бэкенд для ARM Cortex-M и RISC-V микроконтроллеров.
- Пакетный менеджер `apl pkg` для распространения библиотек.
- Драйверы периферии: GPIO, I2C, SPI, UART для дронов и роботов.

Q2 2027

Экосистема & Cloud

- WebAssembly Playground (компилятор YK
- Акторная многопоточность без гонок да
- Международный релиз и первая конфер

БУДУЩЕЕ НАЧИНАЕТСЯ СЕГОДНЯ

Системное программирование может быть надежным и п

YKL устраняет барьер между максимальной производительностью микроконтроллеров и безопасностью памяти.

Открытый исходный код на GitHub:

`github.com/a78722456-cell/YKL-language`

Лицензия: MIT / Apache-2.0 | 100% Rust | Made in Kazakhstan

Спасибо за внимание! Готов ответить на ваши вопросы.